Konfigurator armii w grze strategicznej

Cel: Zaprojektowanie i implementacja w C# systemu do tworzenia i zarządzania jednostkami w fikcyjnej grze strategicznej. System musi elastycznie wspierać różne frakcje, złożone jednostki specjalne (bohaterów) oraz mechanikę klonowania jednostek.

Kontekst biznesowy (Scenariusz): Tworzymy grę strategiczną, w której gracz może dowodzić armiami różnych frakcji. Każda frakcja ma unikalny wygląd i statystyki dla swoich podstawowych jednostek. Dodatkowo, gracz może rekrutować "Bohaterów", których ekwipunek i umiejętności można szczegółowo konfigurować. W grze istnieje również mechanika "magicznego klonowania", pozwalająca na kopiowanie istniejących jednostek.

Wymagania implementacyjne i punktacja

1. Zarządzanie ustawieniami gry (Wzorzec: Singleton) - [Maks. 25 %]

System musi posiadać globalny dostęp do jednego obiektu konfiguracyjnego `GameEngineSettings`.

- **[5 %]** Poprawna implementacja wzorca (prywatny konstruktor, statyczna instancja, statyczna metoda `GetInstance`) .
- **[10 %]** Zapewnienie **bezpieczeństwa wątkowego** (prawidłowe użycie `lock` na dedykowanym obiekcie `_lock`) .
- **[5 %]** Implementacja odporności na refleksję (sprawdzenie `_instance != null` w prywatnym konstruktorze i rzucenie wyjątku) .
- **[5 %]** Zgodność z **Zasadą Pojedynczej Odpowiedzialności (SRP)**. Klasa `GameEngineSettings` ma *tylko* zarządzać swoim cyklem życia i przechowywać konfigurację.
    - *Kara:* Umieszczenie w klasie Singleton logiki biznesowej (np. metod obliczających obrażenia) zamiast wydzielenia jej do innych klas, skutkuje utratą **-5 %** .

2. Fabryki frakcji (Wzorzec: Fabryka Abstrakcyjna) - [Maks. 30 %]

Gra posiada dwie frakcje: "Królestwo Północy" (Rycerz, Łucznik) oraz "Imperium Pustyni" (Pustynny Wojownik, Skorpion Bojowy).

- **[10 %]** Poprawne zdefiniowanie wszystkich wymaganych abstrakcji (interfejsy `IUnitFactory`, `IInfantryUnit`, `IMechanicalUnit`) .
- **[10 %]** Poprawna implementacja dwóch konkretnych fabryk (`NorthernKingdomFactory`, `DesertEmpireFactory`) .
- **[5 %]** Poprawna implementacja konkretnych klas produktów (np. `Knight`, `Archer` itd.).
- **[5 %]** Poprawna implementacja metody współpracy (np. `TeamUp`), która demonstruje kompatybilność produktów z tej samej fabryki .
- **Wymóg (Zasada Odwrócenia Zależności - DIP):** Kod kliencki (wywołujący) musi zależeć *wyłącznie* od abstrakcji (interfejsów), a nie od konkretnych fabryk czy produktów.

- *Kara:* Stworzenie w kodzie klienckim "na sztywno" instancji konkretnej fabryki (np. `new NorthernKingdomFactory()`) zamiast polegania na abstrakcji `IUnitFactory` skutkuje utratą **-10 %**.

3. Konstruktor Bohatera (Wzorzec: Budowniczy) - [Maks. 25 %]

Bohater (`Hero`) jest obiektem złożonym (imię, hełm, zbroja, broń, lista umiejętności).

- **[5 %]** Poprawne zdefiniowanie produktu (`Hero`) oraz interfejsu budowniczego (`IHeroBuilder`) .
- **[10 %]** Poprawna implementacja konkretnego budowniczego (`HeroBuilder`) zawierającego logikę konstrukcji obiektu i metodę `Build()` .
- **[10 %]** Implementacja wzorca jako **Fluent Builder** (metody ustawiające zwracają instancję budowniczego lub kolejny interfejs, pozwalając na łączenie wywołań) .
- **Wymóg (Hermetyzacja):** Obiekt `Hero` po zbudowaniu powinien być niemutowalny (lub trudny do modyfikacji z zewnątrz). Proces konstrukcji musi być w pełni kontrolowany przez Budowniczego.
  - *Kara:* Zastosowanie publicznych setterów w klasie `Hero` lub pozostawienie publicznego konstruktora, co pozwala na tworzenie obiektu z pominięciem budowniczego, skutkuje utratą **-5 %**.

4. Klonowanie jednostek (Wzorzec: Prototyp) - [Maks. 20 %]

Wszystkie jednostki w grze (z fabryk i Bohaterowie) muszą być klonalne.

- **[5 %]** Zdefiniowanie wspólnego interfejsu/metody klonowania dla jednostek (np. `Clone()` w `IGameUnit`).
- **[5 %]** Poprawna implementacja **płytkiej kopii** (`ShallowCopy`) z wykorzystaniem `MemberwiseClone` .
- **[10 %]** Poprawna implementacja **głębokiej kopii** (`DeepCopy`) dla klasy `Hero`, która poprawnie kopiuje typy referencyjne (np. tworzy nową, niezależną instancję `List<string> Skills`) .
- **Wymóg (Poprawność logiki):** Należy wykazać zrozumienie różnicy między kopią płytką a głęboką.
  - *Kara:* Błędna implementacja kopii głębokiej (np. skopiowanie tylko referencji do listy `Skills` zamiast utworzenia nowej listy) skutkuje utratą **-10 %**.

Dołączony kod zadania

Dołączona solucja zawiera dwa projekty:

1. Projekt realizujący logikę biznesową: ma ustawienia , tworzy jednostki frakcji , bohatera i próbuje klonować. Jednak robi to w sposób skrajnie nieelastyczny.
2. Projekt z implementacją testów jednostkowych które powinny działać kiedy system będzie zaimplementowany w sposób zgodny z wymaganiami. Aktualnie wszystkie testy są zakomentowane ponieważ się nie kompilują. Podczas wprowadzania zamian z kolejnego podpunktu należy je sukcesywnie odkomentowywać i uruchamiać dokumentując poprawność wykonanych zadań.

Wdrażanie Wzorców Projektowych

Zadanie polega na przekształceniu powyższego chaosu w profesjonalny system konfiguracji armii.

Twoim celem jest przekształcenie powyższego kodu w profesjonalny moduł `ArmyConfigurator`. Wykonaj poniższe kroki, korzystając z mechanizmów faktoryzacji:

Krok 1: Wdrożenie wzorca Singleton

Zmień `GameSettings` w bezpieczny `GameEngineSettings`.

- Stwórz prywatny, statyczny obiekt `_lock` do obsługi współbieżności.
- Zaimplementuj metodę `GetInstance()` z blokadą `lock`.
- W prywatnym konstruktorze rzuć wyjątek, jeśli `_instance` nie jest nullem (ochrona przed refleksją).

Krok 2: Wdrożenie wzorca Fabryka Abstrakcyjna

Zamiast `UnitManager` z instrukcjami `if`, stwórz hierarchię fabryk.

- Zdefiniuj interfejsy `IInfantryUnit` oraz `IMechanicalUnit`.
- Stwórz `IUnitFactory` z metodami `CreateInfantry()` i `CreateMechanical()`.
- Zaimplementuj fabryki: `NorthernKingdomFactory` i `DesertEmpireFactory`.

Krok 3: Wdrożenie wzorca Budowniczy (Fluent Builder)

Zhermetyzuj proces tworzenia klasy `Hero`.

- Uczyń pola klasy `Hero` prywatnymi (lub `readonly`).
- Stwórz `HeroBuilder`, w którym metody takie jak `SetName()` zwracają `this` (Fluent Interface).
- Dodaj metodę `Build()`, która zwraca gotowy obiekt `Hero`.

Krok 4: Wdrożenie wzorca Prototyp

Napraw mechanikę klonowania jednostek.

- Dodaj interfejs `IGameUnit` z metodą `Clone()`.
- W klasie `Hero` zaimplementuj **głębokie kopiowanie** (Deep Copy) – musisz stworzyć nową instancję listy `Skills`, zamiast kopiować tylko referencję.

Jak sprawdzić poprawność?

W kodzie klienckim (metoda `Main`) nie powinieneś widzieć ani jednego słowa kluczowego `new` w odniesieniu do jednostek (użyj fabryki) ani bohatera (użyj buildera). Zmiana imienia sklonowanego bohatera nie powinna zmieniać imienia oryginału.

Kryteria oceny

- **Poprawność implementacji:** Czy każdy z 4 wzorców został użyty zgodnie z jego przeznaczeniem i założeniami z instrukcji?
- **Zgodność z SOLID:** Czy kod (szczególnie Fabryka Abstrakcyjna i Budowniczy) przestrzega zasad DIP i SRP?
- **Obsługa przypadków brzegowych:** Czy Singleton jest bezpieczny wątkowo i odporny na refleksję? Czy Prototyp poprawnie rozróżnia kopię płytką od głębokiej dla typów referencyjnych?
- **Jakość kodu:** Czytelność, nazewnictwo, hermetyzacja.